ElderPing Platform — Architecture & Technical Documentation

Table of Contents

---

1. Platform Overview

ElderPing is a cloud-native healthcare platform hosted entirely on AWS. It is built for three types of users — **Elders**, **Caregivers**, and **Admins** — and provides features including health monitoring, medication reminders, emergency alerts, AI-powered health insights, appointment scheduling, and a financial operations (FinOps) dashboard for super admins.

The platform runs on **Amazon EKS** (Kubernetes 1.31) across **2 Availability Zones** and uses a **GitOps** deployment model powered by **ArgoCD**. All infrastructure is provisioned and managed by **Terraform**.

---

2. End-to-End Request Flow

There are two types of requests: **UI requests** (serving the frontend app) and **API requests** (serving data to the frontend).

UI Request Flow (Static Frontend)

```
User Browser
  → Route53 (DNS resolution for elderping.online)
  → CloudFront CDN
  → S3 Bucket (React SPA — static HTML, JS, CSS)
  ← Response served back via CloudFront edge cache
```

The React frontend is a Single Page Application (SPA) deployed to an S3 bucket. CloudFront acts as the CDN, caching the static assets at edge locations globally. Origin Access Control (OAC) ensures that S3 content is only accessible through CloudFront — the S3 bucket itself is not publicly accessible.

API Request Flow

```
User Browser / Mobile App
  → Route53 (DNS: api.elderping.online)
  → ALB (Application Load Balancer) — WAF Web ACL attached
  → NGINX Ingress Controller (inside EKS)
  → Kubernetes Service (ClusterIP)
  → Microservice Pod (e.g., auth-service, notes-service)
  → RDS PostgreSQL (if data read/write needed)
  → External AWS services via VPC PrivateLink (SNS, SES, SQS, Bedrock, etc.)
  ← JSON response returned back through the same chain
```

ArgoCD UI Request Flow

```
User Browser
  → Route53 (argocd.elderping.online)
  → ALB
  → NGINX Ingress Controller
  → ArgoCD Server pod (argocd namespace)
```

---

3. DNS & Entry Point

Amazon Route53 is the DNS layer for the `elderping.online` domain. It routes traffic based on subdomain:

Subdomain	Destination	Purpose
`elderping.online` / `www`	CloudFront	Serves the React SPA frontend
`api.elderping.online`	ALB	All microservice API traffic
`argocd.elderping.online`	ALB	ArgoCD web UI

All Route53 records are **Alias A records** pointing to the ALB DNS name or CloudFront distribution. Alias records have no additional DNS query cost and support health checks natively.

---

4. WAF — Why on ALB and Not CloudFront

This is a common question. Here is the precise answer for ElderPing:

CloudFront handles only the **UI** (`elderping.online`) — it serves static HTML/JS/CSS files from S3. There is no dynamic API traffic going through CloudFront.

All API traffic (`api.elderping.online`) goes directly through **Route53 → ALB**, bypassing CloudFront entirely. This means:

- If WAF were only on CloudFront, all API requests would be completely unprotected.
- By attaching WAF to the ALB, every API call — regardless of origin — is inspected before reaching any pod.

The Terraform resource that wires this together:

```
resource "aws_wafv2_web_acl_association" "alb" {
  resource_arn = module.alb.alb_arn
  web_acl_arn  = module.waf.web_acl_arn
}
```

WAF on ALB protects against: SQL injection, XSS, rate limiting, known bad IPs, and OWASP Top 10 attacks — all before the request reaches any Kubernetes pod.

---

5. VPC & Networking

The entire platform runs inside a single VPC (`10.0.0.0/16`) spanning **2 Availability Zones** (AZ-a and AZ-b).

Subnet Layout

Subnet Type	CIDR	Contents
Public Subnet AZ-a	`10.0.1.0/24`	ALB node
Public Subnet AZ-b	`10.0.2.0/24`	ALB node
Private Subnet AZ-a	`10.0.3.0/24`	EKS worker nodes, RDS Primary
Private Subnet AZ-b	`10.0.4.0/24`	EKS worker nodes, RDS Standby

VPC PrivateLink Endpoints

All communication between EKS pods and AWS services (ECR, S3, Secrets Manager, CloudWatch, SNS, SQS, SES, KMS) goes through **VPC PrivateLink Interface Endpoints**. This means:

- No traffic leaves the AWS network (no NAT Gateway hops for AWS API calls)

- Lower latency for AWS service calls
- Better security posture — pods never need internet access for AWS APIs

---

6. EKS Cluster — Namespaces & Pods

The EKS cluster uses **namespaces** to isolate workloads by function and apply different security policies to each group.

Namespace: `healthcare`

Purpose: Runs all application microservices. This namespace has the strictest Pod Security Standard (`restricted`) — pods cannot run as root, must drop all Linux capabilities, and must use a seccomp profile.

Every microservice runs with **2 replicas** (minimum) and scales up to **5 replicas** via Horizontal Pod Autoscaler (HPA) at 80% CPU utilization.

Pod	Port	Responsibility
`auth-service` (×2)	3000	JWT authentication, user login/signup, superadmin bootstrap, role-based access
`health-service` (×2)	3000	Elder health records, vitals tracking, linked elder relationship checks
`notes-service` (×2)	3000	Caregiver notes with PostgreSQL storage, linked elder access control
`reminder-service` (×2)	3000	Medication and appointment reminders
`alert-service` (×2)	3000	Emergency alerts sent via SNS
`ai-service` (×2)	3000	AI health queries via AWS Bedrock (Amazon Nova Lite), voice check-in summaries, FinOps recommendations
`finops-service` (×2)	3000	Cost dashboard, AWS cost metrics, optimization recommendations for super admins
`appointment-service` (×2)	3000	Appointment scheduling between elders, caregivers, and providers
`audit-service` (×2)	3000	Audit log records for compliance
`notification-service` (×2)	3000	Push and email notifications via SNS and SES

Pod	Port	Responsibility
`report-service` (×2)	3000	Patient report generation, stores output to S3
`frontend` (×2)	80	React SPA served in-cluster (CloudFront/S3 is the primary path)

Total application pods (minimum): 24 pods (12 services × 2 replicas each)

Each pod has:

- Liveness probe at `/healthz`
- Readiness probe at `/ready`
- CPU limit: 500m / request: 100m
- Memory limit: 512Mi / request: 128Mi
- Runs as non-root user (UID 1000)
- Read-only root filesystem where applicable
- Pod Disruption Budget: minimum 1 pod always available during node disruptions

---

Namespace: `platform`

Purpose: Runs cluster-level infrastructure components. Uses `baseline` Pod Security Standard — more permissive than `healthcare` because infrastructure operators sometimes need elevated permissions.

Pod	Responsibility
`argocd-server`	ArgoCD web UI and API
`argocd-application-controller`	Watches GitOps repo, reconciles cluster state
`argocd-repo-server`	Clones and renders Helm charts from Git
`argocd-dex-server`	SSO/OIDC login for ArgoCD
`argocd-redis`	ArgoCD's internal cache (not application cache)
`aws-load-balancer-controller`	Provisions ALB from Kubernetes Ingress resources

Namespace: `security`

Purpose: Runs the External Secrets Operator (ESO). Uses `baseline` Pod Security Standard.

Pod	Responsibility
`external-secrets`	Controller that syncs AWS Secrets Manager → Kubernetes Secrets
`external-secrets-webhook`	Validates ExternalSecret CRD manifests
`external-secrets-cert-controller`	Manages TLS certificates for the webhook

The ESO service account (`external-secrets`) has an IAM role attached via IRSA (IAM Roles for Service Accounts). This role has `secretsmanager:GetSecretValue` permission for the `elderpinq/*` path in AWS Secrets Manager.

Namespace: `monitoring`

Purpose: Runs the full observability stack. Uses `privileged` Pod Security Standard because Prometheus node exporters need host-level access to collect system metrics.

Pod	Responsibility
`prometheus-server`	Scrapes metrics from all pods via ServiceMonitor CRDs
`grafana`	Dashboards for metrics visualization (10Gi persistent volume)
`loki`	Log aggregation — stores logs from all pods (50Gi persistent volume)
`promtail` (DaemonSet)	Runs on every node, ships pod logs to Loki
`alertmanager`	Routes Prometheus alerts to notification channels
`kube-state-metrics`	Exposes Kubernetes object metrics to Prometheus
`node-exporter` (DaemonSet)	Exposes host-level metrics (CPU, memory, disk) per node

---

Namespace: `elderping`

Purpose: Legacy/utility namespace for miscellaneous cluster resources not tied to the healthcare namespace.

---

7. External Secrets Operator — What It Is & How It Works

External Secrets Operator (ESO) is a Kubernetes controller that bridges AWS Secrets Manager and Kubernetes. Without ESO, you would have to manually create Kubernetes Secrets and update them every time a secret rotates. ESO automates this entirely.

How It Works — Step by Step

Step 1 — ClusterSecretStore (one-time setup)

A `ClusterSecretStore` resource is deployed to the cluster. It tells ESO: "connect to AWS Secrets Manager in `us-east-1`, and authenticate using the `external-secrets` service account."

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
  name: elderpinq-secrets-store
spec:
  provider:
    aws:
      service: SecretsManager
      region: us-east-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets
            namespace: security
```

The `external-secrets` service account has an IAM role attached via IRSA annotation. No static AWS credentials are used.

Step 2 — ExternalSecret (per service)

Each Helm chart has an `ExternalSecret` manifest. This tells ESO exactly which AWS Secrets Manager keys to pull and what Kubernetes secret keys to create them under. Example for auth-service:

```
data:
  - secretKey: db-password
    remoteRef:
      key: elderpinq/dev/db
      property: password
  - secretKey: jwt-secret
    remoteRef:
      key: elderpinq/dev/jwt
      property: secret
  - secretKey: superadmin-password
    remoteRef:
      key: elderpinq/dev/superadmin
      property: password
```

Step 3 — ESO Syncs the Secret

ESO reads the `ExternalSecret`, calls `secretsmanager:GetSecretValue` on each AWS path, and creates/updates a standard Kubernetes `Secret` object in the namespace. It re-syncs every **1 hour** (configurable via `refreshInterval`).

Step 4 — Pod Consumes the Secret

The `Deployment` references the Kubernetes secret as an environment variable:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
```

```
        name: auth-service-secrets
        key: db-password
```

The pod never knows or cares about AWS Secrets Manager — it just sees an environment variable.

Why ESO Instead of Hardcoding Secrets

- Secrets are never stored in Git
- Secrets rotate in AWS Secrets Manager without redeploying code
- Access is audited via CloudTrail (every `GetSecretValue` call is logged)
- Single source of truth — one change in AWS SM propagates to all pods within 1 hour

---

8. Secrets Flow — From AWS to Pod

```
Developer/Admin
  → Creates secret in AWS Secrets Manager
     path: elderpinq/{env}/{secret}
     e.g.: elderpinq/dev/db → { "password": "..." }

  → ESO controller (running in security namespace)
     reads ExternalSecret CRD every 1 hour
     calls AWS SM GetSecretValue via IRSA (no static keys)

  → ESO creates/updates Kubernetes Secret
     in the same namespace as the pod
     e.g.: auth-service-secrets

  → Kubernetes injects secret into pod as env var
     e.g.: DB_PASSWORD, JWT_SECRET, SUPER_ADMIN_PASSWORD

  → Pod reads env var at startup
     no hardcoded credentials anywhere in code or Git
```

---

9. AI Service — Deep Dive

What It Does

The `ai-service` provides three AI-powered capabilities:

1. **General Health Queries** — answers symptom checks, medication questions, risk assessments, and general health Q&A
2. **Voice Check-In Summaries** — receives voice-transcribed text from an elder, summarizes it into a clinical note, and saves it via the notes-service
3. **FinOps Recommendations** — analyzes infrastructure billing data and suggests cost optimizations for EKS, RDS, and Bedrock

AI Model Used

Amazon Bedrock — Amazon Nova Lite (`amazon.nova-lite-v1:0`)

Nova Lite is Amazon's lightweight multimodal model. It was chosen for:

- Native AWS integration (no external API calls, stays within VPC via PrivateLink)
- Low cost: `$0.00006 per 1K input tokens`, `$0.00024 per 1K output tokens`
- Fast response times for conversational queries
- No need to manage API keys — uses IRSA for authentication

How a Query Flows Through the AI Service

```
Frontend (React)
  → POST /api/ai/ai/query
     { userId, capability, query }

  → ALB → NGINX Ingress → ai-service pod

  → authMiddleware validates JWT token
  → requirePermission checks user has 'AI_EXECUTE' permission
  → checkRelationship verifies user can query on behalf of userId

  → Prompt is constructed:
     "System: You are an intelligent healthcare assistant...
      Scoped capability: {capability}. User query: {query}"

  → providerRegistry.getProvider() returns AwsBedrockProvider

  → BedrockRuntimeClient.send(InvokeModelCommand)
     modelId: amazon.nova-lite-v1:0
     payload: { messages: [{ role: 'user', content: [{ text: prompt }] }],
                inferenceConfig: { maxTokens: 1000 } }

  → Bedrock returns:
     { output: { message: { content: [{ text: "..." }] } },
       usage: { inputTokens: N, outputTokens: M } }

  → Cost is calculated:
     cost = (inputTokens * 0.00006 + outputTokens * 0.00024) / 1000

  → Interaction is logged to PostgreSQL (ai_interactions table):
     user_id, model_id, capability, prompt, response, tokens, cost

  → Response returned to frontend:
     { result: "AI-generated health insight..." }
```

Voice Check-In Flow

```
Elder speaks → frontend transcribes to text
  → POST /api/ai/ai/voice-checkin { userId, transcribedText }

  → ai-service constructs summary prompt:
     "Analyze the following voice checkin transcript...
      Summarize key issues under 150 words. Transcript: '...'"

  → Bedrock Nova Lite generates summary

  → ai-service calls notes-service internally:
     POST http://notes-service:3000/notes
     { userId, noteType: 'AI_NOTE', content: 'AI Generated Voice Check-in
Summary: ...' }
```

```
  → Note is persisted to PostgreSQL by notes-service
  → Summary returned to frontend
```

Provider Architecture

The AI service uses a **provider registry pattern** (`providerRegistry.js`) that abstracts the AI backend. It supports:

- `AwsBedrockProvider` — production provider, calls Amazon Bedrock
- `MockAiProvider` — for local development, returns static responses without calling Bedrock

The registry picks the provider based on environment configuration. This makes it easy to swap models or providers in the future without changing business logic.

AI Interaction Audit Table

Every single AI call — query, voice check-in, FinOps recommendation — is logged to PostgreSQL:

```
CREATE TABLE ai_interactions (
  id               SERIAL PRIMARY KEY,
  user_id          INT NOT NULL,
  model_id         VARCHAR(100),        -- amazon.nova-lite-v1
  capability       VARCHAR(50),         -- health_query, voice_summary,
finops_recs
  prompt_payload   TEXT,
  response_payload TEXT,
  input_tokens     INT DEFAULT 0,
  output_tokens    INT DEFAULT 0,
  estimated_cost   DECIMAL(12,8),       -- in USD
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Super admins can view all AI interactions via `GET /api/ai/ai/interactions`.

---

10. Logging & Observability

Application Logs — Loki + Promtail

Every pod writes logs to stdout/stderr. **Promtail** runs as a DaemonSet on every EKS node and ships those logs to **Loki** (log aggregation). Logs are queryable in **Grafana** using LogQL.

Flow:

```
Pod stdout/stderr
  → Promtail (DaemonSet on node)
  → Loki (50Gi persistent storage in monitoring namespace)
  → Grafana (query & dashboard)
```

Metrics — Prometheus + Grafana

Prometheus scrapes metrics from all pods every 15 seconds using `ServiceMonitor` CRDs (configured via `kube-prometheus-stack` Helm chart). Services expose a `/metrics`

endpoint.

Flow:

```
Pod /metrics endpoint
  → Prometheus scrapes via ServiceMonitor
  → Prometheus stores time-series data
  → Grafana reads via PromQL queries
  → Dashboards & Alerts displayed
```

kube-state-metrics exposes Kubernetes object state (pod restarts, deployment status, HPA scaling events) to Prometheus.

node-exporter exposes host-level metrics (CPU, memory, disk I/O) per node.

CloudWatch — AWS-Level Logs

EKS control plane logs (API server, audit, scheduler) and AWS service logs flow to **CloudWatch Log Groups**. Retention is set to **90 days** (configured via `log_retention_days` variable).

CloudTrail — API Audit

Every AWS API call — including `secretsmanager:GetSecretValue`, ECR pulls, S3 access, and IAM operations — is recorded in **CloudTrail**. This provides a complete audit trail for compliance.

How Logs Are Fetched in Practice

Log Type	How to Access
Pod application logs	Grafana → Explore → Loki → filter by namespace/pod
Infrastructure metrics	Grafana → Dashboards → Prometheus data source
AWS API calls	AWS Console → CloudTrail → Event History
EKS control plane	AWS Console → CloudWatch → Log Groups → `/aws/eks/...`
Alerts	Alertmanager routes to configured receivers (Slack, email, etc.)

---

11. CI/CD Pipelines

Pipeline Order: Terraform First, Then Application

Infrastructure must exist before application deployments. The pipelines are designed to run in this order.

Swim Lane 1 — Terraform CI Pipeline (Infrastructure as Code)

Trigger: Pull request or push to `main`/`develop` branches when files in `environments/**` or `modules/**` change.

Authentication: GitHub Actions uses **OIDC** — no static AWS access keys are stored. The runner assumes an IAM role via `role-to-assume: ${{ secrets.AWS_ROLE_ARN }}`.

```
Code Push (infra change)
  │
  ▼
Stage 1 — Checkov Security Scan
  Uses: bridgecrewio/checkov-action@v12
  Scans Terraform code for misconfigurations (open S3 buckets, public RDS,
unencrypted storage, etc.)
  soft_fail: true — reports findings but never blocks the pipeline
  │
  ▼
Stage 2 — Terraform Plan
  1. OIDC auth → assume IAM role
  2. terraform init (pulls providers, connects to S3 backend for state)
  3. terraform validate (syntax check)
  4. terraform plan (reads current state from S3, computes diff — READ ONLY, no
changes applied)
  5. Plan output posted as PR comment for human review
  │
  ▼
Stage 3 — Infracost Cost Estimation
  Reads Terraform plan and estimates monthly AWS cost for any resource changes
  Breakdown posted as PR comment
  API Key stored in GitHub Secrets as INFRACOST_API_KEY
  │
  ▼
Stage 4 — Simulated Apply
  echo "Apply complete. Resources: 138 added, 0 changed, 0 destroyed."
  No actual terraform apply runs — this protects all currently running
infrastructure
  Real applies require the separate terraform-apply.yml workflow (manual trigger
only)
```

State Management:

- Terraform state stored in S3: `elderpinq-462355914183-tfstate/dev/terraform.tfstate`
- State locking via DynamoDB table: `elderpinq-tfstate-lock`
- `terraform plan` is read-only — it reads state but never writes it

Swim Lane 2 — Application CI/CD Pipeline

Trigger: Push to the application's main branch in its own repo.

```
Developer pushes code to e.g. elderping-auth-service repo
  │
  ▼
GitHub Actions CI (in the service's own repo)
  1. Build & Test (unit tests, lint)
  2. Docker Build (Dockerfile in repo root)
```

```
  3. OIDC auth → Push Docker image to Amazon ECR
     Image tagged with Git SHA: arunnsimon/elderpinq-auth-service:{sha}
  │
  ▼
CD step: Update GitOps repo
  GitHub Actions commits to elderping-gitops repo:
  charts/auth-service/values.yaml → image.tag: {new-sha}
  │
  ▼
ArgoCD detects change (polls GitOps repo every 3 minutes)
  ArgoCD renders Helm chart with new image tag
  ArgoCD applies the new Deployment to EKS
  Kubernetes performs rolling update:
    - Starts new pod with new image
    - Waits for readiness probe to pass
    - Terminates old pod
    - Repeats per pod
  │
  ▼
Application updated with zero downtime
```

ArgoCD App-of-Apps Pattern: A single root ArgoCD application watches the `argocd/applications/` directory in the GitOps repo. Every `.yaml` file in that directory is an ArgoCD Application manifest. ArgoCD auto-discovers and manages all of them. Adding a new service is as simple as adding a new YAML file — no manual ArgoCD configuration needed.

---

12. Terraform & tfvars

Are tfvars Files Used?

Yes. The file is at `environments/dev/terraform.tfvars` and contains:

```
aws_region         = "us-east-1"
environment        = "dev"
domain_name        = "elderping.online"
kubernetes_version = "1.31"
log_retention_days = 90
db_password        = "SuperSecurePassword456!"
```

Is the tfvars File Pushed to Git?

No. The `.gitignore` in the infra repo explicitly excludes it:

```
*.tfvars
*.tfvars.json
```

This means the `terraform.tfvars` file exists locally and in the developer's environment but is never committed to GitHub.

How Does the Terraform CI Pipeline Run Without tfvars in Git?

The Terraform CI pipeline (`terraform-ci.yml`) runs `terraform plan` without a `-var-file` flag. Terraform uses the following fallback chain:

1. Environment variables with the `TF_VAR_` prefix (e.g., `TF_VAR_db_password`)

2. Auto-loaded `terraform.tfvars` file (not present in CI runner — file is gitignored)
3. Variable defaults defined in `variables.tf`

For the CI pipeline, this means:

- Non-sensitive variables (`aws_region`, `environment`, `domain_name`) have defaults in `variables.tf`
- Sensitive variables like `db_password` either have a default or need to be set as a GitHub Secret and injected as `TF_VAR_db_password` in the workflow

The `terraform plan` in CI is **read-only** — it shows what would change without applying anything. Even if a variable uses its default value, the plan is still valid for review purposes.

For a real `terraform apply` (done manually via `terraform-apply.yml`), the operator runs locally with the actual `terraform.tfvars` file present.

---

13. Cache Service

ElderPing does **not currently use a dedicated application-level cache service** (no ElastiCache Redis or Memcached module exists in the Terraform configuration).

The only Redis instance running in the cluster is **ArgoCD's internal Redis** (`argocd-redis` pod in the `platform` namespace). This is used exclusively by ArgoCD for its own state caching and is not accessible to application pods.

Each microservice connects directly to RDS PostgreSQL for data. If caching is needed in the future, ElastiCache for Redis would be the natural addition — it would fit between the microservice pods and RDS to cache frequently read data (e.g., user sessions, health records).

---

14. Why NGINX Ingress Controller

The cluster uses the **AWS Load Balancer Controller** (which provisions the ALB from Ingress resources) combined with **NGINX Ingress Controller** for in-cluster routing. Here is why NGINX was chosen over alternatives:

NGINX Ingress vs Alternatives

Feature	**NGINX Ingress**	**kgateway (Envoy Gateway)**	**AWS ALB Ingress only**
Maturity	Very high — industry standard since 2015	Newer, less production-proven	High but limited features
Path-based routing	Full support	Full support	Limited annotation-based
Header manipulation	Full support	Full support	Limited
Rate limiting	Built-in	Built-in	Not supported

Feature	NGINX Ingress	kgateway (Envoy Gateway)	AWS ALB Ingress only
WebSocket support	Yes	Yes	Yes
gRPC support	Yes	Yes	Partial
Community & docs	Largest ecosystem	Growing	AWS-specific
Kubernetes certification	Official	CNCF project	AWS-maintained
Learning curve	Low	Medium–High	Low

Why Not kgateway

kgateway (formerly Envoy Gateway) implements the Kubernetes Gateway API spec. While it is the future direction of Kubernetes ingress, it was newer and less battle-tested at the time ElderPing's infrastructure was designed. NGINX Ingress has a proven track record in production healthcare workloads.

Why Not ALB Ingress Only

Using only the AWS Load Balancer Controller means every service needs its own ALB (or complex annotation-based sharing). NGINX Ingress provides a single in-cluster proxy that the ALB forwards to — this reduces ALB costs and gives fine-grained control over routing rules inside the cluster.

How It Works in ElderPing

The ALB receives traffic → forwards to NGINX Ingress Controller pod → NGINX routes to the correct Kubernetes Service by path prefix → ClusterIP Service → Pod.

Each service's `values.yaml` defines its path:

```
ingress:
  className: "alb"
  hosts:
    - host: api.elderping.online
      paths:
        - path: /api/auth
          pathType: Prefix
```

---

15. Supporting AWS Services

Service	Purpose in ElderPing
SNS	Sends SMS and push notifications for emergency alerts (alert-service) and reminders (reminder-service)
SQS	Async message queues between services — decouples producers from consumers for non-blocking operations

Service	Purpose in ElderPing
SES	Sends transactional emails (appointment confirmations, report ready notifications) from the `elderping.online` domain
EventBridge	Scheduled rules (cron) that trigger Lambda on a schedule for automated report generation
Lambda	Runs the report generation job — triggered by EventBridge, writes output PDF/data to the S3 patient reports bucket
S3 (patient-reports)	Stores generated patient reports. Accessed via VPC PrivateLink (no public internet)
ECR	Private Docker image registry for all 12 microservice images
AWS Secrets Manager	Single source of truth for all secrets — pulled by ESO into Kubernetes Secrets
Amazon Bedrock	Hosts Amazon Nova Lite model — called by ai-service for all AI inferences

---

16. Security

Security Tools in Use

Tool	Where	Purpose
AWS WAF	Attached to ALB	Blocks OWASP Top 10 attacks, SQL injection, XSS, rate limiting before requests reach pods
CloudTrail	AWS account level	Records every AWS API call — provides full audit trail for compliance
AWS Config	AWS account level	Continuously evaluates AWS resources against compliance rules — flags misconfigurations
Pod Security Standards	Kubernetes namespace level	`restricted` on healthcare namespace — no root, no privilege escalation, seccomp enforced
Network Policies	Kubernetes	Controls which pods can talk to which other pods — enforced per namespace
IRSA (IAM Roles for Service Accounts)	EKS	Each pod/controller gets a scoped IAM role — no shared credentials, no static keys

Tool	Where	Purpose
OIDC for GitHub Actions	CI/CD	Pipelines never store AWS access keys — assume role dynamically via OIDC trust
Pod Disruption Budgets	Kubernetes	Minimum 1 pod always running — prevents all replicas being evicted simultaneously

Security Tools Removed from Diagram

GuardDuty, Inspector, and Security Hub were part of the original Terraform configuration but are not highlighted in the simplified diagram. They operate at the AWS account level automatically once enabled and do not require application-level integration.